

Machine Learning and Presentation

due: Dec 2 if you are presenting on Dec 3rd
due: Dec 8 if you are presenting on Dec 9th or Dec 12th

Sameer Faisal, Ray Koo

University of California, Riverside

sfais004@ucr.edu, rkoo002@ucr.edu

Training a Machine Learning Model

Sample code for the following can be found [here](#).

- Train the given methods based on your Machine Learning Task:
 - **Regression:** Artificial Neural Networks (ANN), Linear Regression (LR) and polynomial regression models from above code.
 - **Classification:** Convolutional Neural Networks (CNN) and K-Nearest Neighbors (KNN).
 - **Clustering:** Kmeans and Kmeans++.
 - **Timeseries Data (classification):** Long Short Term Memory (LSTM).
 - **We used the Classification requirements, so KNN, CNN (See the code in Colab).**

Hyper-parameter Tuning

- For each model, tune key hyperparameters and visualize their effect:
 - **Classification:**
 - * CNN: Number of filters, learning rate, number of epochs, activation functions; plot accuracy vs. epochs and discuss convergence epoch. For CNN, we found that when we changed the number of epochs from 50 to 100, that yielded the most positive effect on the model out of all our tunings in the code, improving the F1 score and recall the most. We also found that when we changed other parameters such as batch size, and activation function to tanh, it was also affecting these scores in a positive way, but not as much as the changing the amount of epochs. Finally, we saw that decreasing the learning rate especially, but also increasing the size of the neural network, in turn, affected the F1 and Recall scores in a negative way compared to our original CNN model. So, overall, we found that changing the amount of epochs from 50 to 100 resulted in the best F1 and Recall scores out of all our hyper parameter tunings.
 - * KNN: In our code, we specified the number of neighbors to be 51. For distance metrics we used, manhattan, euclidean, and minkowski. We used the accuracy vs k neighbors to specify which metric yielded the best results. As a result, we found that Manhattan had the best accuracy at 0.9627. Finally, we also found that an elbow point occurred in all three of these metrics at the exact same point, k=4.

Evaluating your model

- After training and tuning hyperparameters, evaluate your model's performance and summarize key observations:
 - * Classification: For KNN, we found that although the accuracy and precision were high, the Recall was dangerously low at 0.58, meaning there were not as many many true positives in our training data as we wanted there to be. Also our F1 score being at 0.72 also shows that we cannot really rely too much on this model to predict whether a person is a diabetic as there are a good amount of false diagnoses as pointed out in the Confusion Matrix. Now for CNN, after hyperparameterizing tuning, we found that the F1 score and Recall levels went up to 0.89 and 0.84 respectively. This shows that the CNN model did an overall better job in predicting whether a person is diabetic as it is pointed out by the Recall score. Furthermore, this model is shown to be more reliable in that there are less false predictions as pointed out by the F1 score and the Confusion Matrix. As a result, we can see that the CNN model can consistently predict more reliably on whether a person has diabetes as the F1 score and Recall are significantly higher than in the KNN model (These observations stood out to us but you can look over our google Colab for accuracy, precision).

Dimensionality Reduction

Our data is not solely configured of numerical data, there is categorical data as well such as smoking history and gender, which leads us to not being able to perform PCA.

Project Presentation and Scheduling

- Please choose the time your group will be presenting from the [this link](#) and add all your group members names.
- Please use the following slides template for your project (you can use google slides or local slides): [slides template](#). Attach PDF of your slides to this submission.